AGV KIM2A — System Reference

Oct 22, 2026 · @Jo'el Evander Nathanael Irawan

Overview

KIM2A (profile `agv1_kim`) is a differential-drive magnetic-tape AGV run by a Python asyncio controller with a Flask web HMI on its Ubuntu PC. The code lives at github.com/joelevander10/agv-kim2a-controller, branch `main`, and on the AGV in `~/agv-kim2a-controller`.

Item	Value
AGV PC login	`agv1-kim` (SSH)
AGV PC address (AGV network)	192.168.2.100
AGV PC address seen on Wi-Fi	192.168.0.102 (DHCP — can change)
Web HMI	http://<AGV address>:5000
DIO module (DI + DO, Modbus TCP)	192.168.1.31:502, unit 1
AO module (wheel speed, Modbus TCP)	192.168.1.30:502, unit 1
RFID reader (TCP)	192.168.5.10:2022
PLC (Mitsubishi SLMP / MC binary)	192.168.4.10:1280
Magnetic sensor	Roboteq MGS1600 on CAN, node 6, COB-ID 390 (0x186)
Calibration encoder	CANopen friction wheel, node 1, 125 kbit/s
Drive wheels	0.18 m diameter, gearbox 1:30, track 0.46 m
Front sensor ahead of axle	0.076 m (placeholder — measure and update)
USB keypad	name contains "XING WEI"

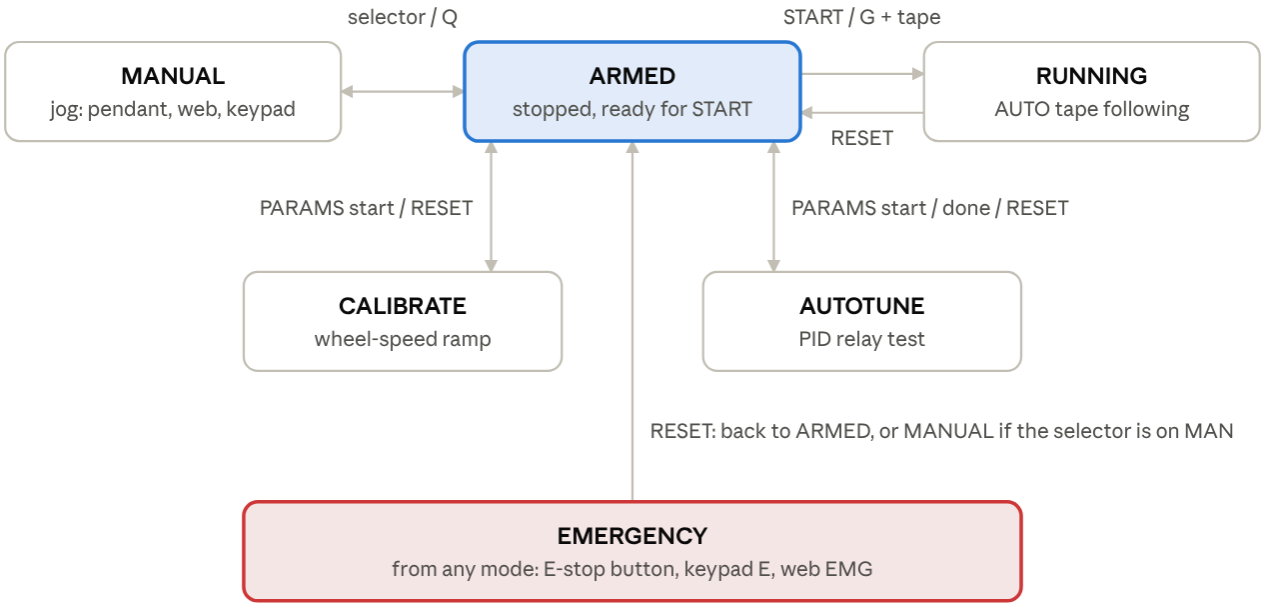
Run it: `cd ~/agv-kim2a-controller && AGV_ID=agv1_kim python3 main.py`. The profile is `profiles/agv1_kim.json`; `agv2_kim.json` currently fails to load (missing `DECEL_RATE`).

Main files: `main.py` (start-up, drivers), `modes.py` (mode manager, AUTO and MANUAL loops), `config.py` (loads the profile), `core/pid.py`, `core/sequence_engine.py` (RFID sequences), `keyboard_control.py` + `drivers/keypad_evdev.py` (keypad), `autotune.py`, `app/app.py` (web HMI).

Operating modes

The AGV always starts in ARMED; from there the selector or Q goes to MANUAL, START (with tape under the sensor) goes to RUNNING, and RESET brings every run back to ARMED.

ARMED is the hub: every run starts and ends there



operating modes · 6 states

- **Emergency** stops everything with the motors idled (brakes released, the AGV can be pushed by hand). It latches until RESET, even after the button is released.
- **MANUAL brakes:** whenever nothing is jogging (no pendant button, web button or keypad move key held), both wheel brakes (DO2, DO5) are ON so the AGV holds still; they release on the next jog command.
- **Q override:** Q forces MANUAL or AUTO until someone moves the physical selector; then the selector rules again.
- **Web button mode** (MANUAL page) replaces the panel START / RESET / selector with on-screen buttons; the physical EMERGENCY still works.
- **System error** (a driver stops answering: DI module, CAN or RFID) brakes the AGV and holds until the driver comes back.

- **PLC status:** MANUAL sets M2003, ARMED/RUNNING set M2004, RUNNING sets M2005, emergency sets M2001/M2002.

IO mapping

The AGV uses 9 of 16 digital inputs, 6 of 16 digital outputs and both analog outputs; DI9–DI15 and DO6–DO15 are free. Channel numbers come from `io_mapping` and `motor_channels` in the profile.

Digital inputs (DIO module 192.168.1.31, `DI_FLIPPED` = 0)

DI	Signal	Contact / logic
0	START	Momentary push button, rising edge starts AUTO from ARMED
1	RESET	Momentary push button, rising edge = reset
2	MODE selector	ON = MANUAL, OFF = AUTO
3	EMERGENCY	ON = emergency active (NO contact)
4	Pendant FORWARD	Hold to jog, MANUAL only
5	Pendant REVERSE	Hold to jog, MANUAL only
6	Pendant LEFT	Hold to jog, MANUAL only
7	Pendant RIGHT	Hold to jog, MANUAL only
8	PROX station marker	Magnetic proximity; rising edge stops the AGV at a trolley station (`marker_side: prox`)

Digital outputs (DIO module 192.168.1.31)

DO	Signal
0	Left motor FORWARD
1	Left motor REVERSE
2	Left motor BRAKE
3	Right motor FORWARD
4	Right motor REVERSE
5	Right motor BRAKE

Motor driver alarm inputs are not wired (`do_alarm` = 888, disabled).

Analog outputs (AO module 192.168.1.30, register base 4096)

AO	Signal	Range
0	Left wheel speed	0–5 V commanded (module range 10 V, 12-bit)
1	Right wheel speed	0–5 V commanded (module range 10 V, 12-bit)

Motor map from the encoder calibration of 2026-05-30: left rpm = 1295 × V − 354.2, right rpm = 1289.8 × V − 299.1 (motor rpm, before the 1:30 gearbox).

CAN bus (CANable adapter): MGS1600 magnetic sensor node 6 with flags tape detect = 2, left marker = 4, right marker = 8, sensor fail = 128; sensor timeout 0.5 s. The calibration encoder shares the adapter and is only used during wheel-speed calibration.

USB keypad

The keypad uses the AGV Kayaba layout; keys for features KIM2A does not have are refused with a log message and do nothing. Hold keys act only while held; one-press keys act on the press.

Key	Action	Type	Works in
W / ↑	Forward	Hold	MANUAL
S / ↓	Reverse	Hold	MANUAL
A / ←	Spin left on the spot	Hold	MANUAL
D / →	Spin right on the spot	Hold	MANUAL
W + A / W + D	Curve forward left / right without stopping (inner wheel at MANUAL SLOW speed)	Hold	MANUAL
S + A / S + D	Curve backwards left / right	Hold	MANUAL
G	START AUTO (same as the START button; needs tape under the sensor)	Press	ARMED
1	AUTO speed APPROACH	Press	ARMED, RUNNING
2	AUTO speed SLOW	Press	ARMED, RUNNING

Key	Action	Type	Works in
4	AUTO speed HIGH	Press	ARMED, RUNNING
Q	Switch MANUAL ↔ AUTO (overrides the selector until it is moved)	Press	Any
K / L	Follow left / right tape track	Press	ARMED, RUNNING
E	EMERGENCY STOP, latched until RESET	Press	Any
ENTER	RESET (same as the RESET button)	Press	Any
Z	PLC pusher EXTEND (M2212)	Hold	MANUAL, SLMP on
X	PLC pusher RETRACT (M2213)	Hold	MANUAL, SLMP on
U / I / O	PLC top conveyor FWD / RVS / CLR ERR (M2104 / M2105 / M2106)	Hold	MANUAL, SLMP on
B / N / M	PLC bottom conveyor FWD / RVS / CLR ERR (M2124 / M2125 / M2126)	Hold	MANUAL, SLMP on
3, 5	MEDIUM / MAX speed — not on KIM2A	—	—
7, 8, 9, 0	Cabang — not on the keypad, pick it on the HMI	—	—
R, T, Y	Auto reverse / rotate — not on KIM2A	—	—

- Driving stops within 0.4 s if the keypad is unplugged or the controller stops hearing it; once stopped in MANUAL the wheel brakes come on.
- PLC bits drop to 0 on release, unplug, or when the AGV leaves MANUAL; opposite keys held together (Z+X, U+I, B+N) cancel.
- Settings: profile block `keypad` — `ENABLED`, `NAME_CONTAINS` ("XING WEI"), `GRAB` (keys go only to the AGV).
- Setup on the AGV PC: `sudo apt install python3-evdev`, `sudo usermod -aG input agv1-kim`, reboot; check with `python3 debugging/list_keypads.py`.

PLC (SLMP) mapping

The AGV talks to the trolley PLC at 192.168.4.10:1280 every 20 ms over SLMP (MC protocol, binary); it writes its status and requests and reads the PLC's inputs and lamps. R = AGV reads, R/W = AGV writes.

AGV status and requests

Device	Meaning	Dir
M0	PB START AUTO	R
M1	MODE: MAN	R
M2	MODE: AUTO	R
M3	EMERGENCY	R
M4	MASTER ON	R
M2000	AGV.READY (not starting up, not emergency)	R/W
M2001	AGV.EMERGENCY	R/W
M2002	AGV.REQ TROLLEY EMERGENCY (follows AGV emergency)	R/W
M2003	AGV.MANUAL	R/W
M2004	AGV.AUTO (ARMED or RUNNING)	R/W
M2005	AGV.RUNNING	R/W
M2010–M2018	AGV.REQ.SEQUENCE.0–8 (1 s pulse)	R/W
M2040–M2048	SEQUENCE.0–8.COMPLETE	R

Trolley manual control (MANUAL mode only; held while the button or key is held)

Device	Meaning	Keypad	Dir
M2212	Pusher EXTEND	Z	R/W
M2213	Pusher RETRACT	X	R/W
M2104	Top conveyor FWD	U	R/W
M2105	Top conveyor RVS	I	R/W
M2106	Top conveyor CLR ERR	O	R/W

Device	Meaning	Keypad	Dir
M2107	Top conveyor BRAKE	—	R/W
M2108	Top conveyor BRAKE LATCH	—	R
M2109	Top conveyor BRAKE RELEASE	—	R/W
M2124	Bottom conveyor FWD	B	R/W
M2125	Bottom conveyor RVS	N	R/W
M2126	Bottom conveyor CLR ERR	M	R/W
M2127	Bottom conveyor BRAKE	—	R/W
M2128	Bottom conveyor BRAKE LATCH	—	R
M2129	Bottom conveyor BRAKE RELEASE	—	R/W

PLC outputs (Y) and inputs (X) — read only, shown as lamps on the TROLLEY page

Device	Meaning
Y0 / Y1 / Y2 / Y3	Top conveyor FWD / RVS / CLEAR ERROR / BRAKE
Y4 / Y5 / Y6 / Y7	Bottom conveyor FWD / RVS / CLEAR ERROR / BRAKE
Y10 / Y11	Bottom pusher RED (+) / RED (−)
Y12 / Y13	Bottom pusher BLACK (−) / BLACK (+)
Y14 / Y15	Top pusher RED (+) / RED (−)
Y16 / Y17	Top pusher BLACK (−) / BLACK (+)
Y20	PL READY
X0	PB START AUTO
X1	SS MAN/AUTO
X2	EPB TROLLEY
X3	Spare
X4 / X5 / X6 / X7	Proximity TOP front-right / rear-right / rear-left / front-left
X10 / X11 / X12 / X13	Proximity BOTTOM front-right / rear-right / rear-left / front-left

Speeds and PID

AUTO runs at HIGH 0.8 m/s, SLOW 0.18 m/s and APPROACH 0.18 m/s, each with its own PD gains; these are the profile values, and anything saved from the PARAMS page overrides them (see PID auto-tune).

Speed mode	Target (m/s)	KP	TD	N	V_RED_COEF	Output clamp (rpm)
HIGH	0.80	2.3	0.30	20	3.6	±1300
SLOW (corners)	0.18	2.6	0.16	12	2.0	±1300
APPROACH (station, guide bar on the right)	0.18	0.45	0.20	10	0.4	+8 into bar / −80 away

Shared setting	Value
Manual HIGH / SLOW speed	1.0 / 0.5 m/s
Acceleration	0.5 m/s²
Deceleration / APPROACH deceleration	3.0 / 0.4 m/s²
Integral TI / deadband / max	25 / 20 mm / 400
Control period DT	0.01 s
Speed-reduction filter / cap	0.15 / 45 %
Sensor limits	reject
Corner feedforward	on in SLOW; left curves, radius 1.5 m, scale 0.85, filter 0.15 in / 0.6 out

The error is e = −(sensor offset), so positive PID output speeds the left wheel and turns right. The tape followed is the LEFT track by default; RFID `set_track` tags or keys K/L switch it.

RFID tags and sequences

13 tags drive the route: 3 speed zones, 5 cabang forks plus a merge, 1 checkpoint stop and 3 trolley stations. The fork and merge tags (00AA–00AE, 00BB) are placeholders to replace with scanned tags. Tags are ignored while the auto-tune runs.

Tag	Sequence	What it does
0014	speed_zone_slow	Speed SLOW
001C	speed_zone_slow_alt	Speed SLOW (second tag)
0016	speed_zone_high	Speed HIGH
00AA	fork_to_point1	Follow RIGHT track — only if CABANG 1 is selected
00AB	fork_to_point2	Follow RIGHT track — only if CABANG 2 is selected
00AC	fork_to_point3	Follow RIGHT track — only if CABANG 3 is selected

Tag	Sequence	What it does
00AD	fork_to_point4	Follow RIGHT track — only if CABANG 4 is selected
00AE	fork_to_point5	Follow RIGHT track — only if CABANG 5 is selected
00BB	fork_merge_revert	Back to LEFT track after the merge (always)
000C	checkpoint_stop	Brake and hold 4 s, then continue
0096	trolley_load_00	APPROACH crawl → stop on PROX (DI8, 45 s timeout) → 1 s → PLC SEQ 0 → wait complete → resume HIGH
0078	trolley_load_01	APPROACH crawl → stop on PROX (30 s) → 1 s → PLC SEQ 1 → wait complete (60 s) → 1 s → resume HIGH
009A	trolley_load_02	SLOW → stop on LEFT tape marker (15 s) → PLC SEQ 2 → wait complete (30 s) → resume HIGH

- **Cabang** is selected on the HOME page (CABANG 1–5, or SEQ 0–4 which also sets it) and is kept through RESET and emergency.
- NAV (speed-zone tags) → SEQ (stops and PLC handshakes) → SLMP (PLC link) can be switched on and off live on the PARAMS page; turning one off turns off the ones after it.
- Add or change a tag in `sequences` in `profiles/agv1_kim.json`; action types are `set_speed`, `set_track`, `sequence_stop`, `wait_marker`, `stop_agv`, `wait_seconds`, `plc_request`, `wait_plc_complete`, `resume`.

Web HMI and API

The HMI has six pages on port 5000, reached from the nav bar at the bottom of each page.

Page	Path	Used for
HOME	`/`	Mode, speed, sensor, RFID, PLC bits, sequence status; set CABANG / SEQ
MANUAL	`/manual`	Jog buttons and arrow keys; WEB BUTTON MODE (virtual START / RESET / MAN-AUTO / EMG)
IO	`/io`	Live DI / DO / AO

Page	Path	Used for
TROLLEY	`/trolley`	PLC pusher and conveyor buttons, X / Y lamps (MANUAL only)
PARAMS	`/params`	Feature toggles, live tuning, wheel calibration, PID auto-tune, restart
ERRORS	`/errors`	Recent warnings and errors

Endpoint	Method	Purpose
`/api/state`	GET	Full AGV state (mode, sensor, PLC, keypad, sequences)
`/api/manual/command`	POST	Jog command (400 ms watchdog), MANUAL only
`/api/keyboard`	POST	One-press keypad action, e.g. `{"key":"enter"}`
`/api/fork_target`	POST	Set CABANG `point_1` – `point_5` or clear
`/api/sequence`	POST	PLC sequence request 0–8 (0–4 also set CABANG 1–5)
`/api/web_button/mode`, `/api/web_button/press`	POST	Web button bypass of the panel buttons
`/api/params`, `/api/params/features`	GET / POST	Profile values; NAV / SEQ / SLMP live toggles
`/api/params/tuning`, `/api/params/tuning/reset`	GET / POST	Live tuning (saved), reset to profile
`/api/autotune`, `/start`, `/stop`	GET / POST	PID auto-tune status and control
`/api/calibration/status`, `/start`, `/stop`	GET / POST	Wheel-speed calibration ramp
`/api/trolley/state`, `/api/trolley/write`	GET / POST	PLC trolley bits
`/api/io`, `/api/errors`, `/api/wifi`	GET	IO table, error log, Wi-Fi signal
`/api/restart`	POST	Restart `agv-controller.service`

PID auto-tune and live tuning

The auto-tune on the PARAMS page measures how the AGV steers on a straight tape and proposes KP / TD / N for HIGH or SLOW; APPLY saves them to `data/tuning_agv1_kim.json`, which overrides the profile after a restart.

1. Put the AGV in ARMED on a straight tape with clear space ahead (about speed × time limit metres).
2. PARAMS → PID AUTO-TUNE: choose HIGH or SLOW, keep Relay 20 rpm, Max error 50 mm, Time limit 30 s, Cycles 4, press START.
3. The AGV accelerates on its current gains, weaves gently for up to 10 s (relay test), then brakes and fits a model.
4. Three gain sets appear: Calm, Recommended, Responsive. Press APPLY on one, then check with a normal AUTO run.

- If it says the relay never steered back, raise Relay (rpm) and use a straighter section.
- RESET, STOP, E, emergency, MANUAL, tape loss or sensor timeout end the test and brake; RFID tags are ignored during it.
- LIVE TUNING on the same page edits speeds, ramps, gains, clamps and feedforward; changes are saved and kept after restart. RESET TO PROFILE deletes the saved file. Tuning is locked while RUNNING, in emergency, or during the auto-tune.
- The measured model uses `kinematics.SENSOR_LOOKAHEAD_M` (axle to front sensor); 0.076 m is a placeholder until measured.

Updating code and troubleshooting

Code changes go laptop → GitHub → AGV: commit and **Push origin** in GitHub Desktop, then on the AGV `cd ~/agv-kim2a-controller && git pull` and restart the controller (`sudo systemctl restart agv-controller.service`, or the RESTART button on PARAMS). Without internet on the AGV, push over SSH instead: `git push agv main` from the laptop (remote `agv` = `agv1-kim@<AGV address>:/home/agv1-kim/agv-kim2a-controller`).

Symptom	Check
Keypad does nothing	`groups` must include `input`; `python3 debugging/list_keypads.py` must list "XING WEI"; the controller must be running; `/api/state` → `keypad_devices`
W/A/S/D don't move	AGV must be in MANUAL (press Q); EMERGENCY needs ENTER/RESET

Symptom	Check
W + A spins instead of curving	Keypad may not send two keys at once — test with `python3 -m evdev.evtest`
START ignored	Tape not under the front sensor (HOME page TAPE lamp)
AGV stops with "LOST TAPE" / "CAN TIMEOUT"	Sensor off the tape, or CAN adapter / MGS1600 link down
SYSTEM ERROR	A driver stopped answering (DI module, CAN or RFID); check cables and the IPs above
PLC keys / TROLLEY buttons do nothing	MANUAL only; SLMP must be ON (PARAMS); PLC at 192.168.4.10 reachable
Fork tag ignored	No CABANG selected, or a different one selected; placeholder tag hex not yet replaced
SSH "host identification has changed"	Address reused by another machine — `ssh-keygen -R <address>` and verify the fingerprint
`git push agv` refused	Files changed on the AGV — commit them there first, then pull on the laptop

Open items: measure `SENSOR_LOOKAHEAD_M`; replace placeholder tags 00AA–00AE and 00BB; fix `agv2_kim.json` (missing `DECEL_RATE`); give the AGV a fixed IP on the Wi-Fi.